



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

A PROJECT PROPOSAL

ON

**MEDISAFE NEPAL: A GNN-BASED APPROACH TO DRUG-DRUG INTERACTION
PREDICTION AND POLYPHARMACY SCREENING**

Submitted By:

Amrit Sapkota	(THA079BCT003)
Kushal Shah	(THA079BCT021)
Lokesh Yadav	(THA079BCT022)
Shishir Timilsina	(THA079BCT040)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

Jun 2026

ACKNOWLEDGEMENT

We would like to thank the Institute of Engineering, Tribhuvan University, for incorporating the major project within the curriculum of Bachelor's in Computer Engineering.

We are especially thankful to our respected Head of Department, **Er. Anup Shrestha**, for his continuous support, valuable suggestions, and encouragement throughout the proposal phase of this project. His guidance has been a source of motivation for our team.

We would like to express our appreciation to the **Department of Electronics and Computer Engineering, Thapathali Campus**, for providing us with the opportunity to undertake this project, which has significantly contributed to strengthening our technical knowledge and problem solving skills.

Amrit Sapkota (THA079BCT003)

Kushal Shah (THA079BCT021)

Lokesh Yadav (THA079BCT022)

Shishir Timilsina (THA079BCT040)

ABSTRACT

Drug-drug interactions (DDIs) happen when two or more medicines taken together change each other's effect, sometimes causing serious harm to patients. In Nepal, there is no widely available digital tool that warns doctors or pharmacists about dangerous drug combinations. This project presents MediSafe Nepal, a web-based system that predicts whether two drugs interact, identifies the specific type of interaction out of 86 possible types, shows how dangerous it is, and tells the user what to do about it. The system uses a graph neural network trained on a large dataset of 1,710 drugs and 192,284 known interactions, combined with a biomedical knowledge graph called Hetionet that provides biological context about drugs, proteins, genes, and diseases. The model learns a numerical representation for each drug by gathering information from neighbouring nodes in the graph, extracts a small subgraph for each drug pair, and refines this subgraph by removing irrelevant connections and adding new connections between drugs that behave similarly. This architecture achieves state-of-the-art performance, reported at 91.53% F1 score and 93.17% accuracy on the DrugBank benchmark. To make the system relevant to Nepal, we map the National List of Essential Medicines (NLEM) Nepal 2021 onto the dataset; of its 398 essential molecules, 324 are relevant to drug-drug interactions, and 196 of these are successfully matched to the benchmark. Each predicted interaction is labelled with a severity level (Critical, Moderate, or Low) and a plain-language recommendation. When a user enters multiple drugs, the system checks every possible pair and ranks them from most dangerous to least dangerous. The system is designed as a web application that can be connected to pharmacy billing software, so that warnings appear automatically when medicines are entered during billing.

Keywords: *Drug-Drug Interaction, DrugBank, Essential Medicines, Graph Neural Network, GraphSAGE, Hetionet, Knowledge Graph, Nepal, Pharmacy Decision Support, Polypharmacy*

TABLE OF CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ABBREVIATIONS	vi
LIST OF SYMBOLS	vii
1. INTRODUCTION	1
1.1. Background	1
1.2. Motivation	2
1.3. Problem Definition	3
1.4. Objectives	3
1.5. Scope and Applications	4
2. LITERATURE REVIEW	6
2.1. Review of Related Works	6
2.2. Comparative Summary of Reviewed Works	9
2.3. Research Gap	9
3. METHODOLOGY	11
3.1. System Block Diagram	11
3.2. Description of Working Principle	13
3.3. Flowcharts and Algorithms	22
3.4. Instrumentation Tools	24
4. EXPECTED OUTCOME	26
5. PROJECT SCHEDULE	27
6. PROJECT BUDGET	28
7. FEASIBILITY ANALYSIS	29
7.1. Technical Feasibility	29
7.2. Economic Feasibility	30
7.3. Operational Feasibility	30
7.4. Schedule Feasibility	31

7.5. Legal Feasibility	31
REFERENCES	32

LIST OF FIGURES

Figure 3-1. System Block Diagram of MediSafe Nepal	11
Figure 3-2. Merging DDI Graph with Hetionet Knowledge Graph into Combined Network	15
Figure 3-3. GraphSAGE Message Passing: Each Node Collects Information from Neighbours	17
Figure 3-4. Drug-Flow Subgraph Extraction for a Target Drug Pair (h, t)	18
Figure 3-5. Knowledge Subgraph Generation: Irrelevant Edges Removed, Similarity Edges Added	20
Figure 3-6. Overall System Flowchart	23
Figure 5-1. Project Timeline (Gantt Chart)	27

LIST OF TABLES

Table 2-1. Comparative Summary of DDI Prediction Methods on DrugBank Benchmark	9
Table 3-1. Model and Training Hyperparameters	21
Table 3-2. Software Tools	24
Table 3-3. Hardware Tools	24
Table 3-4. Datasets Used	25
Table 6-1. Project Budget	28

LIST OF ABBREVIATIONS

ACC	Accuracy
ADR	Adverse Drug Reaction
API	Application Programming Interface
AUPRC	Area Under the Precision-Recall Curve
AUROC	Area Under the Receiver Operating Characteristic Curve
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DDI	Drug-Drug Interaction
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GNN	Graph Neural Network
GPU	Graphics Processing Unit
HIV	Human Immunodeficiency Virus
IoU	Intersection over Union
JSON	JavaScript Object Notation
KG	Knowledge Graph
LSTM	Long Short-Term Memory
MLP	Multi-Layer Perceptron
NLEM	National List of Essential Medicines
ReLU	Rectified Linear Unit
REST	Representational State Transfer
RNN	Recurrent Neural Network
SMILES	Simplified Molecular Input Line Entry System
t-SNE	t-Distributed Stochastic Neighbor Embedding
UI	User Interface
WHO	World Health Organization

LIST OF SYMBOLS

$A_{h,t}^{(\tau)}$	Connection-strength (adjacency) tensor at iteration τ
$C_{h,t}^{(\tau)}$	Connection-strength score between node pairs
$H_{h,t}^{(\tau)}$	Node-embedding matrix of the subgraph at iteration τ
K	Number of hops for the enclosing subgraph
L	Number of GraphSAGE layers
P	Maximum relational path length
T	Number of subgraph-refinement iterations
$W_a^{(l)}$	Learnable aggregation weight matrix at layer l
$W_c^{(l)}$	GraphSAGE concatenation weight matrix at layer l
W_r	Relation-specific weight matrix
W_c	Final classifier weight matrix
α	Structure-blending parameter in subgraph refinement
$\delta(\cdot)$	Edge-softmax normalisation function
γ	Edge-pruning threshold in subgraph refinement
$\hat{y}_{h,t}$	Predicted probability distribution over interaction types
$\mathcal{E}_{\text{DDI}}$	Set of drug–drug interaction edges
$\mathcal{E}$	Set of edges in the combined network
$\mathcal{G}_{\text{DDI}}$	Drug–drug interaction graph
$\mathcal{G}_{\text{HKG}}$	Hetionet knowledge graph
$\mathcal{G}$	Combined network
$\mathcal{L}$	Cross-entropy training loss
$\mathcal{R}_{\text{DDI}}$	Set of DDI relation types
$\mathcal{R}$	Set of relation types in the combined network
$\mathcal{S}_{h,t}$	Drug-flow subgraph for target drug pair (h, t)
$\mathcal{V}_{\text{DDI}}$	Set of drug nodes in the DDI graph
$\mathcal{V}$	Set of nodes in the combined network

τ	Subgraph-refinement iteration index
$a_v^{(l)}$	Aggregated neighbourhood summary of node v at layer l
$e_v^{(l)}$	Embedding of node v at GraphSAGE layer l
g_v	Final generic embedding of node v
h_r	Learnable relation embedding for relation r
h_v	Refined node embedding of node v
h_S	Subgraph embedding (mean of node embeddings)
$y_{h,t}$	One-hot ground-truth interaction type

1. INTRODUCTION

1.1. Background

A drug-drug interaction (DDI) happens when two or more medicines taken together change each other's effect. One drug may make another stronger, weaker, or create a completely new and harmful side effect. DDIs are one of the most common and most preventable causes of harm to patients. According to research, DDIs account for about 1% of hospitalizations in the general population and 2 to 5% of hospital admissions in the elderly [1, 2]. A well known example is that if warfarin and aspirin enter the body together, they compete for binding to plasma proteins, which can result in acute bleeding in patients [3].

Discovering drug interactions through laboratory studies and clinical trials is extremely expensive and slow. There are tens of thousands of approved drugs, which means hundreds of millions of possible pairs that cannot all be tested in a lab. The latest DrugBank database with 14,931 drug entries contains only 365,984 known DDI records, which is less than 1% of all potential interactions [4]. This makes the problem a natural fit for machine learning. If drugs and their biological relationships are represented as a graph where each drug is a node and each interaction is an edge, a model can learn the patterns behind known interactions and predict the unknown ones.

In recent years, graph neural networks (GNNs) have been developed for DDI prediction. These models work directly on graph structured data, allowing each drug node to gather information from its neighbours such as proteins, genes, enzymes and other drugs. More advanced methods combine the DDI graph with external biomedical knowledge graphs like Hetionet to provide richer biological context. The current state-of-the-art approach uses knowledge subgraph learning, where a small subgraph is extracted for each drug pair and then refined by removing irrelevant edges and adding similarity edges between drugs that behave alike [3]. This architecture achieves state-of-the-art results, reported at 91.53% F1 score and 93.17% accuracy on the DrugBank benchmark, outperforming all other compared methods.

1.2. Motivation

We selected this project because drug interaction related harm is a serious and preventable problem in Nepal, yet there is no digital tool available to help doctors and pharmacists check for dangerous combinations.

In Nepal, it is very common for people to walk into a pharmacy and buy medicines directly without any prescription or doctor's advice. Antibiotics, painkillers, antacids and blood pressure tablets are handed over the counter on request. A customer may already be taking one medicine at home, then buy a second one for a different complaint, with nobody checking whether the two are safe together. This self-medication culture means that dangerous drug combinations are created every day, completely outside the view of any doctor.

Even in hospitals, a single doctor often sees a very large number of patients in one day. Outpatient departments are crowded and consultation time per patient is short. Under such pressure, it is humanly very easy to forget that a newly prescribed drug might interact with another drug the patient is already taking, especially when the patient is elderly and already on several medicines.

Many patients, particularly older people with chronic diseases like diabetes, hypertension and heart disease, take several medicines at the same time. This is called polypharmacy. A patient on five medicines has ten possible drug pairs to worry about. On eight medicines, twenty eight pairs. No busy clinician can check all of these by memory. Most hospital pharmacies and private clinics in Nepal do not have any computerised interaction checking system. In developed countries pharmacy software automatically shows a warning when a risky combination is dispensed. In Nepal this safety net is largely missing.

Because of all the above, patients in Nepal have suffered serious harm including bleeding, kidney damage, dangerous heart rhythm changes and severe hypoglycemia that trace back to drug combinations that should never have been given together. Every such case

represents a failure not of intention but of information. This motivated us to build a system that can provide this missing information at the point of care.

1.3. Problem Definition

Nepal lacks an affordable, locally relevant and easy to use system that can instantly tell a doctor, pharmacist or patient whether a set of medicines taken together is dangerous, what kind of harm could result and what to do about it. The specific problems are as follows:

First, self-medication is widespread and there is no safety check when patients buy multiple drugs over the counter. Second, doctors are overloaded and cannot mentally cross-check every possible interaction for every patient. Third, polypharmacy is common in elderly patients and the number of possible drug pairs grows rapidly with each additional medicine. Fourth, most pharmacies in Nepal have no digital interaction checking tool. Fifth, identifying DDIs through lab studies is too expensive and slow to cover all possible drug combinations.

This project addresses these problems by using a knowledge-subgraph-enhanced graph neural network that can predict interactions between drug pairs, classify each interaction into one of 86 specific types, assign a severity level and provide a plain language recommendation. The system is tailored to Nepal by mapping the National List of Essential Medicines (NLEM) Nepal 2021 onto the prediction model, so that drugs relevant to Nepal are prioritised for the user, while the model retains its full predictive capability.

1.4. Objectives

The main objectives of this project are:

- I. To build a clinical polypharmacy screening system that can predict two medicines' interaction type and severity.
- II. To make a drug-drug interaction knowledge base specific to Nepal's official medicine list (NLEM 2021).

- III. To deliver the system as a web application that doctors, pharmacists, and patients can use with convenience.

1.5. Scope and Applications

This project focuses on predicting drug-drug interactions using a graph neural network trained on the DrugBank benchmark dataset enriched with the Hetionet biomedical knowledge graph. The model is trained on the full global dataset of 1,710 drugs and 192,284 interactions so that it learns general patterns from maximum available data. The Nepal-specific prioritisation happens at the application layer, where the 196 NLEM-mapped drugs form the prioritised essential set; the underlying model still supports all 1,710 benchmark drugs, and interactions involving non-NLEM drugs are flagged rather than hidden.

The system predicts interactions and classifies them into 86 specific types. Each interaction is labelled with a severity level and a recommendation message. When a user enters multiple drugs the system checks all possible pairs and ranks them by danger. The system also flags drugs that are not present in NLEM Nepal 2021. The output is displayed through a web interface and an API endpoint is provided for integration with pharmacy billing software.

The scope of this project is limited to pairwise drug-drug interactions. The system does not model true three or more way interactions as data for such interactions barely exists. It also does not cover drug-food interactions or drug-disease interactions. The system is a research prototype and is not clinically validated for real hospital deployment. Of the 398 molecules in NLEM Nepal 2021, 74 are excluded as agents without drug-drug interaction potential (vaccines, medical gases, oral rehydration salts and similar), leaving 324 DDI-relevant molecules; of these, 196 are successfully mapped to the dataset and 128 could not be matched, mostly specialised agents absent from the benchmark. The 196 mapped drugs cover the most commonly prescribed and co-prescribed medicines in Nepal.

The system can be applied as a decision support aid for doctors to cross check prescriptions, a safety check for pharmacists before dispensing, an awareness tool for patients practising self-medication, and a teaching aid for pharmacy and medical students learning about drug interactions.

2. LITERATURE REVIEW

This chapter discusses previous studies related to drug-drug interaction prediction, graph neural networks, knowledge graph based methods and biomedical link prediction systems. It is done to review and understand what methods have already been used and what limitations still exist. These works helped us decide what approach should be best for multi-relational DDI prediction. It also identifies the research gap that our proposed system aims to fill.

2.1. Review of Related Works

2.1.1. Traditional Embedding Methods

Bordes et al. [5] proposed TransE, one of the earliest methods for learning embeddings of entities and relations in knowledge graphs. TransE represents each relation as a translation vector between head and tail entity embeddings. While simple and efficient, TransE achieved only 18.32% F1 score and 64.60% accuracy on the DrugBank DDI benchmark [3], making it one of the weakest performers. The main limitation is that TransE uses a shallow linear model that cannot capture complex interaction patterns between drugs.

Yao et al. [6] proposed MSTE, which improves upon TransE by encoding multidirectional semantics of relations for polypharmacy side effect prediction. MSTE achieved 53.96% F1 score on DrugBank, a significant improvement over TransE. However, like all traditional two-stage methods, MSTE learns embeddings and trains the classifier separately, which limits the model's ability to jointly optimize for the prediction task.

Network embedding methods such as DeepWalk [7], node2vec [8] and LINE [9] use neural networks to capture structural information from the DDI graph. DeepWalk achieved 91.77% AUROC on TWOSIDES, showing that structural information alone can be useful. However, these methods do not leverage any external biological knowledge beyond the DDI graph itself, which limits their ability to predict interactions for drugs with few known connections.

2.1.2. GNN-Based Methods Without External Knowledge

Velickovic et al. [10] proposed Graph Attention Networks (GAT), which use attention mechanisms to weigh the importance of each neighbour during message passing. When applied to DDI prediction, GAT achieved only 35.05% F1 score and 78.02% accuracy on DrugBank [3]. This poor performance shows that attention alone is not enough when the DDI graph is extremely sparse with known interactions being less than 1% of all possible pairs.

Zitnik et al. [11] proposed Decagon, which models polypharmacy side effects using graph convolutional networks on a multirelational graph of drugs and proteins. Decagon achieved 86.97% accuracy on DrugBank, a clear improvement over GAT. However, Decagon still operates only on the DDI graph without using external biomedical knowledge, which limits its ability to reason about drugs that share biological targets or pathways.

Huang et al. [12] introduced SkipGNN, which adds skip-graph connections to capture higher-order interactions between drugs. SkipGNN achieved 88.04% accuracy and 62.36% F1 score on DrugBank, performing best among GNN methods without external knowledge. However, due to the sparsity of the DDI graph and over-parameterization of deep models, GNN-based methods without external knowledge may not consistently outperform simpler traditional approaches [3].

2.1.3. GNN-Based Methods With External Knowledge Graphs

Lin et al. [13] proposed KGNN, the first method to leverage external knowledge graphs for DDI prediction. KGNN uniformly samples a fixed number of neighbours around each drug from the knowledge graph and aggregates their features through message passing. It achieved 88.30% accuracy on DrugBank. However, KGNN treats all neighbours equally and does not consider which specific drug pair is being evaluated, meaning each drug gets the same representation regardless of its interaction partner.

Su et al. [14] proposed DDKG, which improves upon KGNN by assigning attention weights to sampled neighbours based on molecular similarity calculated from SMILES

representations. This helps the model focus on more relevant neighbours. However, like KGNN, DDKG still produces representations that are not drug-pair-aware.

Teru et al. [15] proposed GraIL, which introduced a new approach by extracting enclosing subgraphs for each target drug pair from the combined DDI and knowledge graph network. This allows the model to learn drug-pair-aware representations where the same drug can be represented differently depending on which other drug it is being compared with. GraIL achieved 89.63% accuracy on DrugBank and ranked third overall.

Yu et al. [16] proposed SumGNN, which extends the subgraph approach by pruning edges based on node features to remove noise from the extracted subgraphs. SumGNN achieved 91.86% accuracy and 86.88% F1 score on DrugBank, ranking second among all compared methods. However, SumGNN prunes edges based on features that are randomly initialized or pretrained on other tasks and then fixes the subgraph structure during learning. It cannot adaptively adjust the structure during training.

Hong et al. [17] proposed LaGAT, which uses link-aware attention to aggregate messages from neighbours. While LaGAT performs well on DrugBank with 88.86% accuracy, it performed poorly on TWOSIDES with only 88.72% AUROC, making it inconsistent across datasets.

2.1.4. Knowledge Subgraph Learning (State of the Art)

Wang et al. [3] proposed a knowledge-subgraph-enhanced graph neural network that achieves state-of-the-art performance on both DrugBank and TWOSIDES benchmarks. The key innovations of this architecture are as follows. First, it runs a GraphSAGE [18] network on the entire combined DDI and knowledge graph network to produce rich generic embeddings for all nodes. Second, for each target drug pair, it extracts a directionally pruned drug-flow subgraph containing only nodes on relational paths from head drug to tail drug. Third, it iteratively refines both the graph structure and node embeddings simultaneously over multiple iterations, where irrelevant edges are pruned based on learned connection strengths and new similarity edges are added between

disconnected drugs with similar embeddings. Fourth, the final knowledge subgraph contains explaining paths that provide transparent reasoning for each prediction.

This architecture achieved 91.53% F1 score, 93.17% accuracy and 91.89% Cohen’s kappa on DrugBank, and 95.43% AUROC, 94.14% AUPRC and 89.54% AP@50 on TWOSIDES, outperforming all 14 compared methods. The architecture also suffers less from sparse knowledge graphs compared to other methods, because the propagated drug similarities compensate for missing information.

2.2. Comparative Summary of Reviewed Works

Table 2-1 presents a structured comparison of the reviewed methods.

Method (Year)	Category	Approach	DrugBank Performance
TransE [5] (2013)	Traditional	Translational embedding of entities and relations	F1: 18.32%, ACC: 64.60%
DeepWalk [7] (2014)	Traditional	Random walk based network embedding	F1: 49.17%, ACC: 62.90%
GAT [10] (2018)	GNN (no KG)	Attention weighted neighbour aggregation on DDI graph	F1: 35.05%, ACC: 78.02%
Decagon [11] (2018)	GNN (no KG)	Graph convolutional network on multirelational drug-protein graph	F1: 56.24%, ACC: 86.97%
SkipGNN [12] (2020)	GNN (no KG)	Skip-graph connections for higher-order interactions	F1: 62.36%, ACC: 88.04%
KGNN [13] (2020)	GNN (w/ KG)	Uniform neighbour sampling from external knowledge graph	F1: 74.08%, ACC: 88.30%
SumGNN [16] (2021)	GNN (w/ KG)	Enclosing subgraph extraction with edge pruning	F1: 86.88%, ACC: 91.86%
LaGAT [17] (2022)	GNN (w/ KG)	Link-aware attention on extracted subgraphs	F1: 83.69%, ACC: 88.86%
KnowDDI [3] (2024)	GNN (w/ KG)	Knowledge subgraph learning with iterative structure refinement	F1: 91.53%, ACC: 93.17%

Table 2-1.: Comparative Summary of DDI Prediction Methods on DrugBank Benchmark

2.3. Research Gap

The reviewed works reveal three main gaps. First, the DDI prediction methods surveyed above are trained and evaluated on global benchmarks and do not consider which drugs are actually used in a particular country; national or regional adaptation is not a focus of

these works. Second, the reviewed systems predict interactions but do not assign severity levels or provide plain-language recommendations that a clinician or pharmacist can immediately act upon. Third, none of the reviewed tools combines multi-relational DDI prediction with polypharmacy screening designed for integration with pharmacy billing software.

In the context of Nepal specifically, we are not aware of any locally available digital DDI checking tool for pharmacies or clinics, nor of any prior work that maps the National List of Essential Medicines (NLEM) Nepal 2021 onto a drug interaction dataset. In current practice, doctors and pharmacists rely largely on memory and printed references to check interactions, which is insufficient given the thousands of possible drug combinations.

The proposed system addresses these gaps by using the state-of-the-art knowledge-subgraph-enhanced GNN [3] as the prediction backbone, mapping NLEM Nepal 2021 onto the DrugBank benchmark dataset, adding severity classification and recommendation messages for all 86 interaction types, building a polypharmacy screening pipeline that checks all pairwise combinations ranked by danger, and delivering everything through a web application designed for pharmacy billing integration.

3. METHODOLOGY

This chapter explains the complete system architecture, working principle, algorithms used and tools required for the MediSafe Nepal drug-drug interaction prediction and polypharmacy screening system. The prediction backbone is the knowledge-subgraph-enhanced graph neural network proposed by Wang et al. [3], which we implement and adapt for the Nepali clinical context; our own contributions are the mapping of Nepal's essential medicines onto this model, the severity and recommendation engine, the polypharmacy screening pipeline and the web application.

3.1. System Block Diagram

The system consists of two main phases: an offline training phase and an online prediction phase. In the offline phase, a knowledge-subgraph-enhanced graph neural network is trained on a large benchmark dataset of known drug interactions combined with a biomedical knowledge graph. In the online phase, a user enters drug names into the web application and the trained model predicts interactions for every drug pair.

Figure 3-1 shows the overall system block diagram.

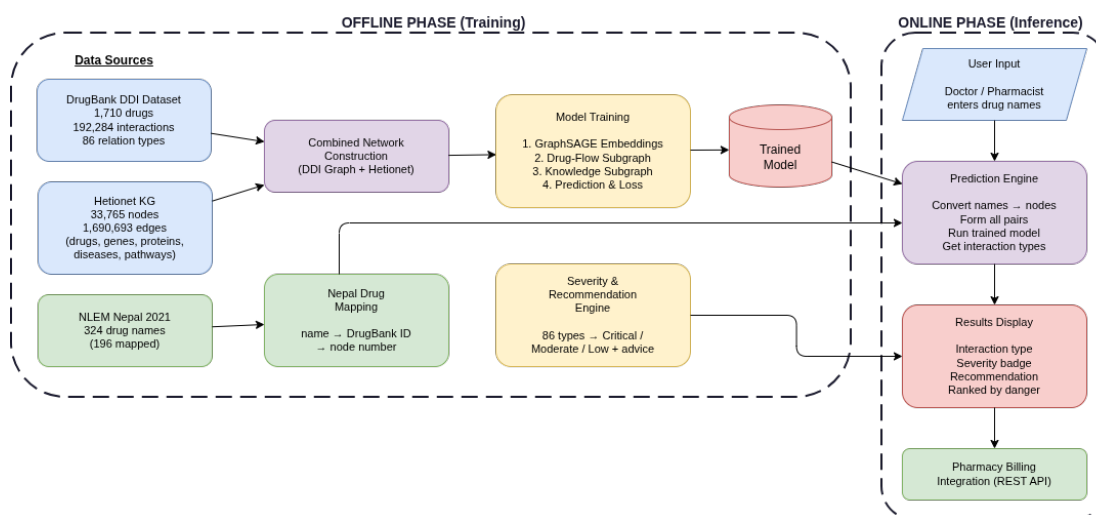


Figure 3-1.: System Block Diagram of MediSafe Nepal

The system block diagram consists of the following major blocks:

Block 1 – Data Sources: Three data sources feed into the system. The DrugBank benchmark dataset provides 1,710 drugs and 192,284 known interactions across 86 relation types. The Hetionet biomedical knowledge graph provides biological context with 33,765 nodes and 1,690,693 edges connecting drugs, genes, proteins, diseases and pathways. The National List of Essential Medicines (NLEM) Nepal 2021 contains 398 essential molecules. Of these, 74 are excluded as agents without relevant drug–drug interaction potential (vaccines, medical gases, oral rehydration salts and similar), leaving 324 DDI-relevant essential molecules that are carried forward for mapping.

Block 2 – Nepal Drug Mapping: The 324 DDI-relevant NLEM molecules are mapped to DrugBank identifiers and then to the internal node numbers used by the dataset. Of these 324 molecules, 196 are successfully matched to nodes present in the DrugBank benchmark and are therefore predictable; the remaining 128 are absent from the benchmark and are recorded as out of scope. This mapping creates three files: a name-to-ID dictionary, a list of Nepal node numbers, and a filtered interaction set for Nepal drugs. It is important to note that the model itself supports all 1,710 nodes in the benchmark; the 196 NLEM-mapped molecules form the prioritised essential core rather than a hard limit on what the model can predict.

Block 3 – Combined Network Construction: Following the KnowDDI preprocessing pipeline [3], the DDI graph and the Hetionet knowledge graph are provided pre-merged as a single large combined network; we load this combined network directly rather than reconstructing it. Drug-drug edges from the validation and test sets are removed to prevent information leakage during training.

Block 4 – Model Training: The knowledge-subgraph-enhanced graph neural network is trained on the combined network using the training set. The model learns to predict the interaction type for any given drug pair.

Block 5 – Severity and Recommendation Engine: All 86 interaction types are manually classified into Critical, Moderate and Low severity levels based on standard clinical

drug-interaction references. Each type is assigned a plain-language recommendation message describing the action a clinician or pharmacist should take.

Block 6 – Web Application: The user enters drug names, the system converts them to node numbers using the Nepal mapping, runs the trained model, retrieves severity and recommendation, and displays results ranked by danger.

3.2. Description of Working Principle

The working principle of the system is divided into two parts: how the model learns to predict interactions (training) and how the system serves predictions to the user (inference).

3.2.1. Training Phase: How the Model Learns

The model architecture follows the knowledge-subgraph-enhanced graph neural network proposed by Wang et al. [3]. We implement this architecture as our prediction backbone; the following five stages describe the KnowDDI method that we adopt. It consists of five stages that work together to predict drug-drug interactions accurately and interpretably.

Stage 1 – Combined Network Construction: A Drug–Drug Interaction (DDI) graph is denoted as

$$\mathcal{G}_{\text{DDI}} = \{\mathcal{V}_{\text{DDI}}, \mathcal{E}_{\text{DDI}}, \mathcal{R}_{\text{DDI}}\},$$

where $\mathcal{V}_{\text{DDI}}$ is the set of drug nodes, $\mathcal{E}_{\text{DDI}}$ is the set of interaction edges, and $\mathcal{R}_{\text{DDI}}$ denotes the set of DDI relation types. Each edge $(u, r, v) \in \mathcal{E}_{\text{DDI}}$ represents an observed interaction between two drugs u and v , where $r \in \mathcal{R}_{\text{DDI}}$ specifies the interaction type (e.g., increased bleeding risk, decreased metabolism, or enhanced toxicity). An external

Hetionet Knowledge Graph (HKG) is incorporated to provide additional biological context and denoted as

$$\mathcal{G}_{\text{HKG}} = \{\mathcal{V}_{\text{HKG}}, \mathcal{E}_{\text{HKG}}, \mathcal{R}_{\text{HKG}}\},$$

where $\mathcal{V}_{\text{HKG}}$ contains biomedical entities such as drugs, genes, proteins, diseases, and biological pathways, $\mathcal{E}_{\text{HKG}}$ represents the relationships among these entities, and $\mathcal{R}_{\text{HKG}}$

denotes the corresponding relation types. The knowledge graph provides rich biological information that extends beyond the observed drug–drug interactions.

Conceptually, the DDI graph and the knowledge graph are merged to form a unified combined network

$$\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathcal{R}\} = \{\mathcal{V}_{\text{DDI}} \cup \mathcal{V}_{\text{HKG}}, \mathcal{E}_{\text{DDI}} \cup \mathcal{E}_{\text{HKG}}, \mathcal{R}_{\text{DDI}} \cup \mathcal{R}_{\text{HKG}}\}.$$

By integrating the two graphs, the model gains access to both known drug interactions and the underlying biological mechanisms connecting drugs to genes, proteins, diseases, and pathways. This enriched representation enables more effective learning of drug interaction patterns and improves DDI prediction performance.

In this work the combined network is not reconstructed from raw sources; for the DrugBank dataset it is provided as part of the benchmark following the KnowDDI preprocessing pipeline and is loaded directly. The resulting graph contains 33,765 nodes and 1,690,693 edges. To prevent information leakage, the drug–drug edges belonging to the validation and test splits are removed before training.

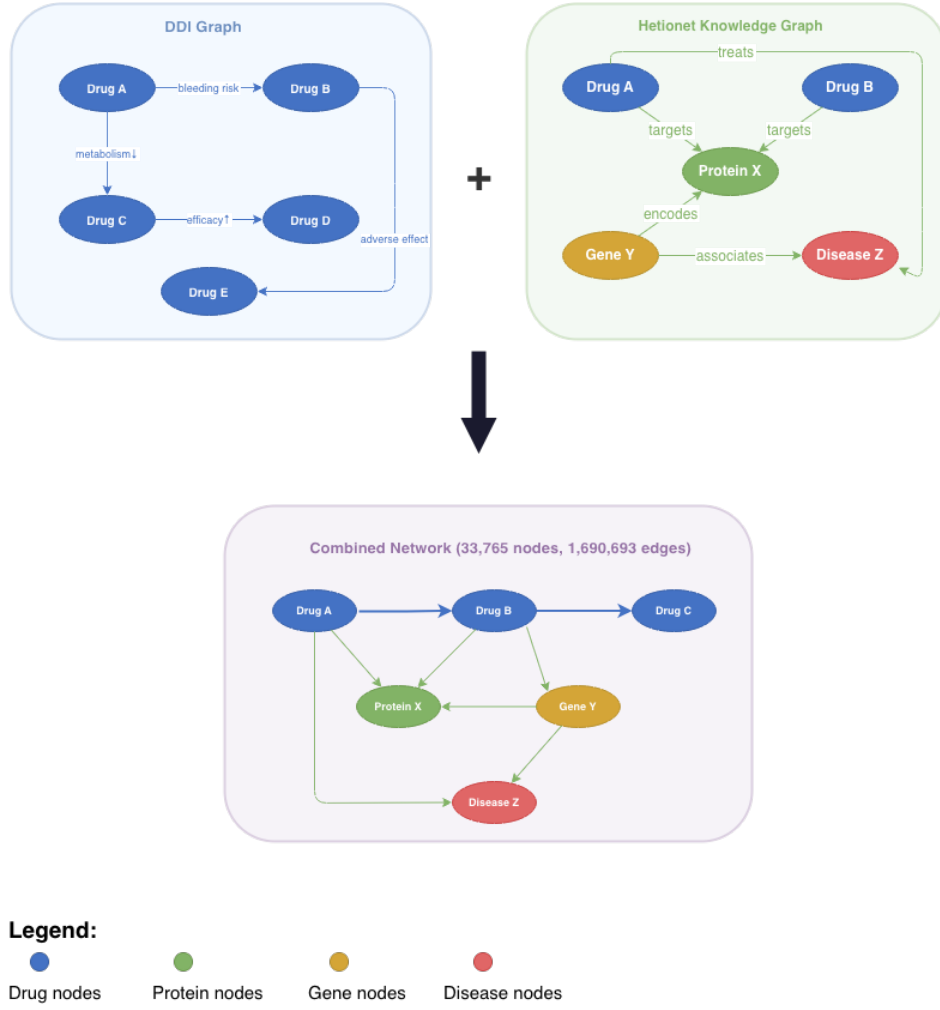


Figure 3-2.: Merging DDI Graph with Hetionet Knowledge Graph into Combined Network

Stage 2 – Generic Embedding Generation via GraphSAGE: To encode the generic knowledge contained in the combined network $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathcal{R}\}$ and enrich the representation of drug nodes with biological context, a two-layer GraphSAGE message-passing network is applied over the entire graph. The objective of this stage is to learn a dense vector representation, referred to as the *generic embedding*, for every node in the combined network. Let $e_v^{(0)}$ denote the initial feature vector of node $v \in \mathcal{V}$.

At the l -th GraphSAGE layer, each node aggregates information from its neighbouring nodes according to

$$a_v^{(l)} = \text{MEAN}\left(\{\text{ReLU}(W_a^{(l)} e_u^{(l-1)}) : (u, r, v) \in \mathcal{E}\}\right), \quad (3-1)$$

where u ranges over the neighbours of v , $\text{MEAN}(\cdot)$ denotes element-wise mean pooling, $W_a^{(l)}$ is a learnable weight matrix, and $\text{ReLU}(\cdot)$ is the Rectified Linear Unit activation function, defined as

$$\text{ReLU}(x) = \max(0, x). \quad (3-2)$$

The resulting neighbourhood summary $a_v^{(l)}$ is then concatenated with the node's embedding from the previous layer and transformed by a second learnable weight matrix:

$$e_v^{(l)} = W_c^{(l)} \left[a_v^{(l)} \parallel e_v^{(l-1)} \right], \quad (3-3)$$

where $W_c^{(l)}$ is a learnable parameter matrix and $\parallel$ denotes vector concatenation along the feature dimension.

The GraphSAGE network consists of two message-passing layers ($L = 2$). After the first layer, each node embedding incorporates information from its immediate (1-hop) neighbours; after the second layer, information is propagated from 2-hop neighbours. Intuitively, this means each drug node captures not only which proteins it targets and which enzymes metabolize it, but also which other drugs share those same proteins and enzymes. The final generic embedding of node v is given by

$$g_v = e_v^{(L)}, \quad L = 2, \quad (3-4)$$

where $g_v \in \mathbb{R}^{32}$ is a 32-dimensional embedding vector.

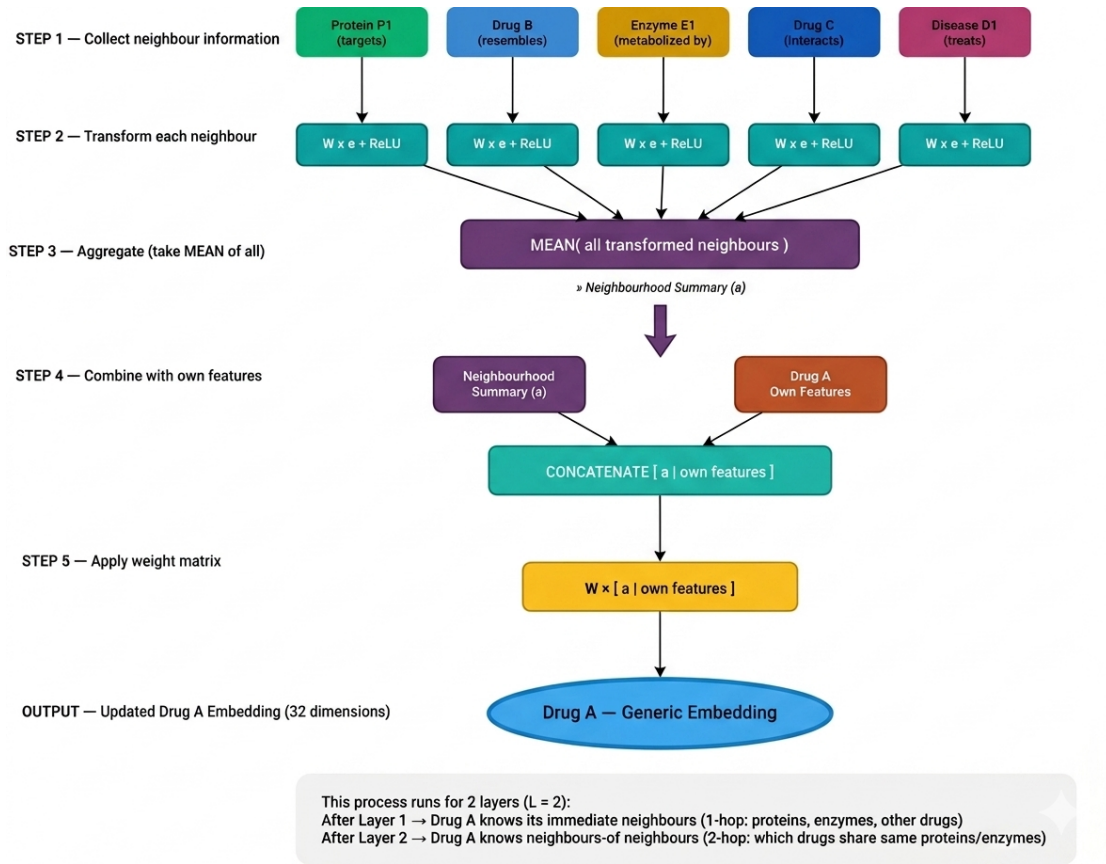


Figure 3-3.: GraphSAGE Message Passing: Each Node Collects Information from Neighbours

Stage 3 – Drug-Flow Subgraph Extraction:

For each drug pair whose interaction the model must predict, a small subgraph is extracted from the combined network. This isolates the local context relevant to that specific pair rather than using the entire massive network. This subgraph is not precomputed; it is generated on demand for each queried drug pair, both during training and during inference.

Formally, given a target drug pair (h, t) , the drug-flow subgraph is denoted

$$\mathcal{S}_{h,t} = \{ \mathcal{V}_{h,t}, \mathcal{E}_{h,t}, \mathcal{R}_{h,t} \},$$

and is extracted in three steps. First, the K -hop enclosing subgraph is taken as the intersection of the K -hop neighbourhoods of h and t (with $K = 2$). Second, all relational

paths pointing from the head drug h to the tail drug t with length at most P (with $P = 4$) are identified, where a relational path has the form

$$h \xrightarrow{r_1} v_1 \xrightarrow{r_2} v_2 \cdots \xrightarrow{r_P} t.$$

A concrete example is: Drug $A \rightarrow$ targets Protein $X \rightarrow$ targeted by Drug $C \rightarrow$ resembles Drug B . Third, directional pruning removes all nodes and edges that do not lie on any such path.

The result is the drug-flow subgraph: a small directed graph containing only nodes and edges that support information flow from the head drug to the tail drug. A direct consequence is that the same drug can be represented differently depending on which other drug it is being compared with.

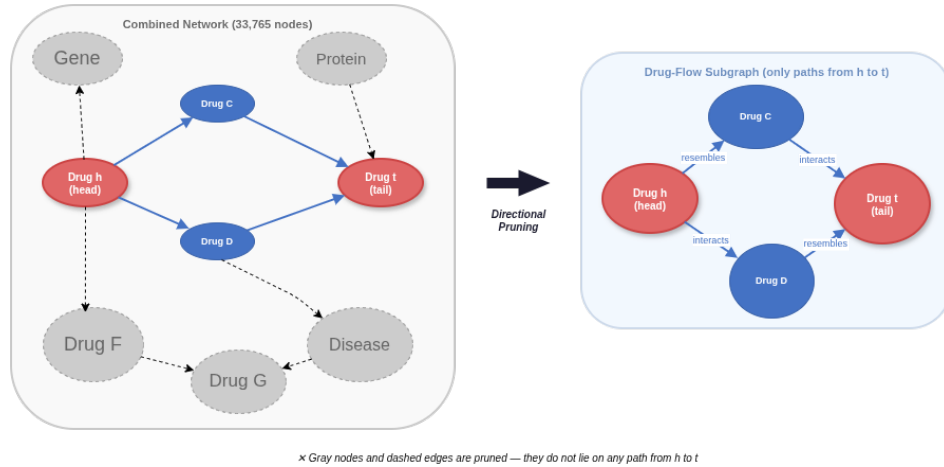


Figure 3-4.: Drug-Flow Subgraph Extraction for a Target Drug Pair (h, t)

Stage 4 – Knowledge Subgraph Generation (Core Innovation):

This is the most important part of the architecture. The drug-flow subgraph $\mathcal{S}_{h,t}$ may still contain irrelevant edges and may be missing useful connections between similar drugs. This stage fixes both problems by iteratively refining the graph structure and node embeddings simultaneously over $T = 3$ iterations.

The node embeddings are initialised from the generic embeddings of Stage 2, i.e. $h_v^{(0)} = g_v$ for every $v \in \mathcal{V}_{h,t}$. Let $A_{h,t}^{(0)}$ denote the initial adjacency of the drug-flow subgraph. At each iteration τ , two steps alternate.

Step A – Estimate Connection Strengths: For every pair of nodes (u, v) in the subgraph and every relation r , a connection strength score is computed by taking the element-wise absolute difference of their embeddings (a measure of dissimilarity), concatenating a learnable relation embedding h_r , and passing the result through a multi-layer perceptron (MLP):

$$C_{h,t}^{(\tau)}(u, v, r) = \text{MLP}\left(\exp(-|h_u^{(\tau-1)} - h_v^{(\tau-1)}|) \parallel h_r\right), \quad (3-5)$$

where $|\cdot|$ denotes the element-wise absolute value and $\parallel$ denotes concatenation. The learned scores are blended with the original structure using a balancing parameter α , and a threshold γ removes weak edges:

$$A_{h,t}^{(\tau)} = \text{ReLU}\left(\delta(\alpha A_{h,t}^{(0)} + (1 - \alpha) C_{h,t}^{(\tau)}) - \gamma\right), \quad (3-6)$$

where $\delta(\cdot)$ is an edge-softmax that normalises the incoming edge weights of each node so that they sum to one. An edge whose score falls below γ is deleted, whereas two previously disconnected nodes with highly similar embeddings are connected by a new edge of relation type “resemble”. The parameter α is selected from the range $[0.3, 0.7]$ and γ from the range $[0.05, 0.2]$.

Step B – Refine Node Embeddings: Using the updated connection-strength tensor as edge weights, relation-aware message passing is performed on the refined subgraph. For each relation r , the corresponding slice of $A_{h,t}^{(\tau)}$ is multiplied with the current node-embedding matrix $H_{h,t}^{(\tau-1)}$ and a relation-specific weight matrix W_r , and the results are averaged across all relations:

$$Q_{h,t,r}^{(\tau)} = \text{ReLU}(A_{h,t}^{(\tau)}(:, :, r) H_{h,t}^{(\tau-1)} W_r), \quad (3-7)$$

$$H_{h,t}^{(\tau)} = \text{MEAN}(\{Q_{h,t,r}^{(\tau)} : r \in \mathcal{R}_{h,t}\}). \quad (3-8)$$

After $T = 3$ iterations, irrelevant edges have been removed, useful similarity edges have been added, and the node embeddings have been transformed to be specifically predictive of the interaction between the target drug pair. The final refined subgraph is called the knowledge subgraph. The new “resemble” edges are the key mechanism by which the model compensates for missing interactions: when two drugs have no

recorded interaction but very similar embeddings, a prediction can propagate through their similarity.

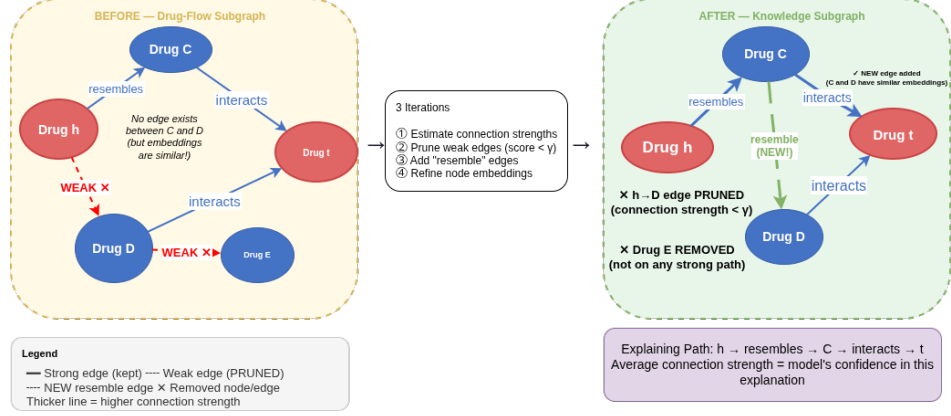


Figure 3-5.: Knowledge Subgraph Generation: Irrelevant Edges Removed, Similarity Edges Added

Stage 5 – Prediction and Classification:

The final prediction is obtained from the knowledge subgraph. First, a subgraph embedding is computed as the mean of all node embeddings in the knowledge subgraph:

$$h_S = \text{MEAN}(\{h_v : v \in \mathcal{V}_{h,t}\}). \quad (3-9)$$

This subgraph embedding is then concatenated with the head-drug embedding h_h and the tail-drug embedding h_t , and the combined vector is passed through a linear classifier with a softmax activation over the 86 interaction types:

$$\hat{y}_{h,t} = \text{softmax}(W_c [h_S \parallel h_h \parallel h_t]), \quad (3-10)$$

where W_c is the classifier weight matrix. The output $\hat{y}_{h,t}$ is a probability distribution over the 86 interaction types, and the type with the highest probability is taken as the predicted interaction. The associated probability is retained as a confidence score, which is later used during inference to distinguish confidently predicted interactions from pairs that are unlikely to interact.

3.2.2. Training Configuration

The model is trained using the configuration reported for the KnowDDI architecture [3], summarised in Table 3-1. The benchmark interactions are split into training, validation and test sets in a 7:1:2 ratio, and the drug–drug edges of the validation and test sets are hidden during training to avoid information leakage. The model is optimised with the Adam optimiser using a cross-entropy loss for multiclass interaction prediction,

$$\mathcal{L} = - \sum_{(h,r,t) \in \mathcal{E}_{\text{DDI}}} y_{h,t}^{\top} \log(\hat{y}_{h,t}), \quad (3-11)$$

where $y_{h,t}$ is the one-hot ground-truth interaction type. Training is stopped early if the validation loss does not improve for ten consecutive epochs.

Table 3-1.: Model and Training Hyperparameters

Hyperparameter	Value
Node embedding dimension	32
GraphSAGE layers	2
Subgraph neighbourhood / path length	2-hop / at most 4
Refinement iterations (T)	3
Structure-blending parameter (α)	0.3 – 0.7
Edge-pruning threshold (γ)	0.05 – 0.2
Optimiser / learning rate	Adam / 5×10^{-3}
Batch size	256
Maximum epochs / dropout	50 / 0.2
Train : validation : test split	7 : 1 : 2

3.2.3. Inference Phase: How the User Gets Results

When a user enters drug names into the web application, the following steps happen:

1. User types drug names (e.g., "Warfarin", "Aspirin") into the web interface.
2. The Nepal drug mapping file converts names to DrugBank IDs (e.g., DB00682, DB00945).
3. The node mapping file converts DrugBank IDs to internal node numbers.

4. The trained model predicts the interaction type (e.g., type 27) together with a confidence score.
5. If the confidence is below a set threshold, the pair is reported as “no significant interaction detected”; otherwise the relation mapping file converts the type (e.g., type 27) to its description, such as "Risk of bleeding can be increased."
6. For a confidently predicted interaction, the severity engine attaches the severity level (e.g., CRITICAL) and recommendation message.
7. Results are displayed to the user with all pairs ranked by severity, and non-interacting pairs grouped separately below them.

Because the multiclass model always assigns one of the 86 interaction types, it does not by itself output a “no interaction” class. To avoid raising a false alarm for every pair, the system applies a threshold on the model’s prediction-confidence score: pairs whose highest predicted probability falls below this threshold are reported as “no significant interaction detected” rather than as a forced interaction label, and are grouped separately below the Critical, Moderate and Low results. This reduces alert fatigue and keeps the user’s attention on the genuinely risky combinations. The threshold is a tunable parameter chosen on the validation set.

If a user enters a drug that is not part of the NLEM mapping, the application flags it with the message "This drug is not listed in Nepal’s National List of Essential Medicines (NLEM 2021)." However, the predicted interaction is still shown rather than suppressed, because NLEM is an essential-medicines policy list rather than a list of all drugs legally available in Nepal; hiding such an interaction could conceal a genuine risk. If a drug is valid but absent from the dataset entirely, the application states that interaction data is not available, since the model has no node from which to make a prediction.

3.3. Flowcharts and Algorithms

3.3.1. Overall System Flowchart

Figure 3-6 shows the complete flowchart of the MediSafe Nepal system from user input to displayed result.

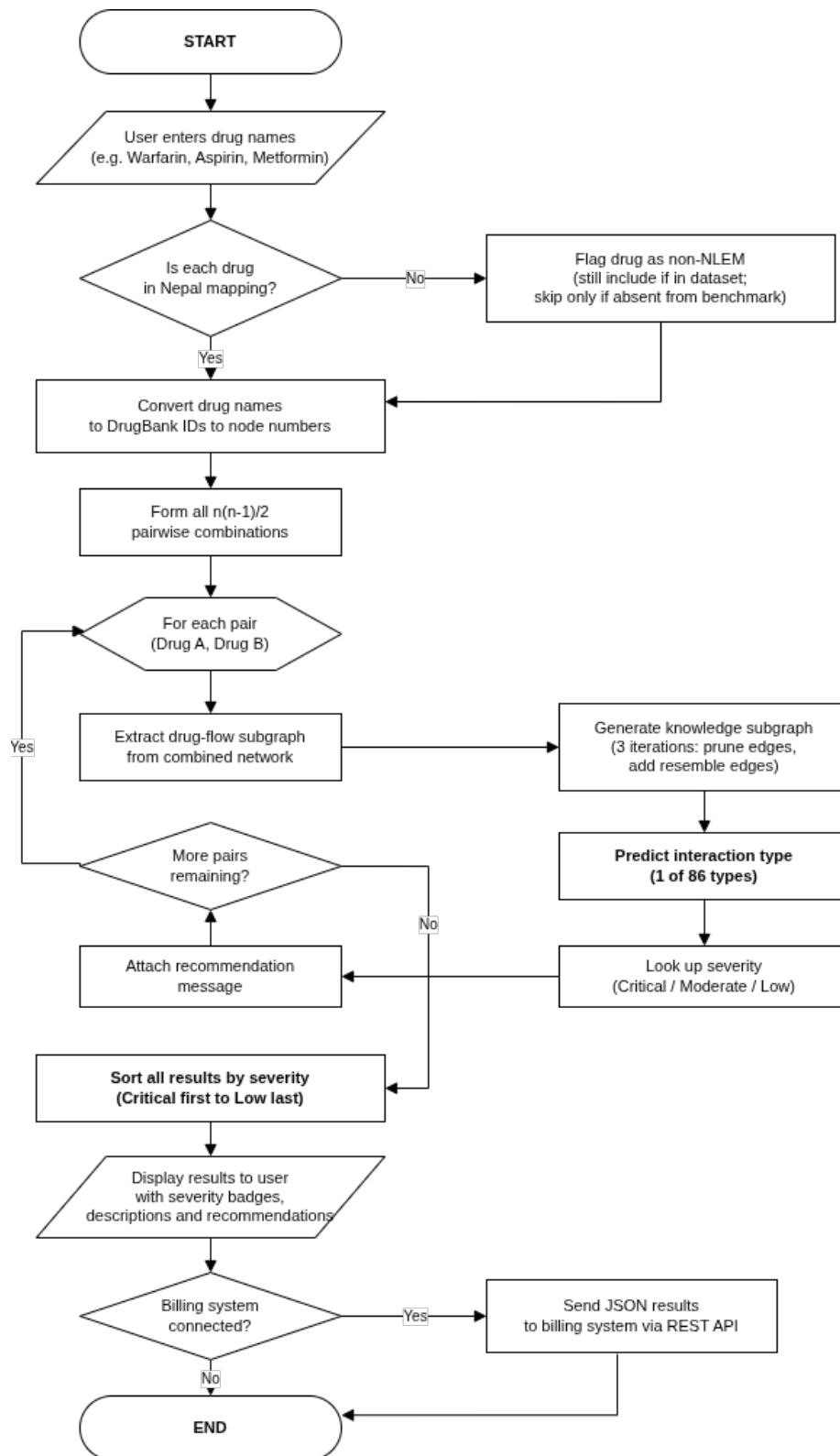


Figure 3-6.: Overall System Flowchart

3.4. Instrumentation Tools

3.4.1. Software Tools

Table 3-2 lists all software tools used in this project.

Table 3-2.: Software Tools

Category	Tools
Programming Language	Python 3.10+
Graph Deep Learning	PyTorch, PyTorch Geometric (PyG)
Data Processing	NumPy, Pandas, NetworkX
Visualization	Matplotlib, Seaborn, PyVis
Web Interface	Streamlit (front-end prototype)
API Framework	FastAPI (REST endpoint for billing integration)
Version Control	Git, GitHub
Development Environment	Google Colab (training), VS Code (development)
Operating System	Ubuntu Linux / Windows

3.4.2. Hardware Tools

Table 3-3 lists the hardware used for training and deployment.

Table 3-3.: Hardware Tools

Component	Specification
GPU (Training)	NVIDIA Tesla T4 (15 GB VRAM) via Google Colab
RAM	Minimum 8 GB system RAM; 16 GB recommended
Storage	At least 5 GB for dataset, model checkpoints and dependencies
CPU (Inference)	Any modern multi-core processor for web application

3.4.3. Datasets

Table 3-4 summarizes all datasets used in this project.

Table 3-4.: Datasets Used

Dataset	Description	Size
DrugBank Benchmark	Processed DDI dataset with 86 interaction types	1,710 drugs, 192,284 interactions
Hetionet	Biomedical KG with drugs, genes, proteins, diseases, pathways	33,765 nodes, 1,690,693 edges
NLEM Nepal 2021	Nepal's official essential medicines list (DDI-relevant subset mapped to the benchmark)	398 molecules (324 DDI-relevant, 196 mapped)

4. EXPECTED OUTCOME

After the completion of this project, the following outcomes are expected:

- I. **Trained DDI Prediction Model:** A fully trained knowledge-subgraph-enhanced graph neural network capable of predicting drug-drug interactions and classifying each interaction into one of 86 clinically distinct relation types.
- II. **Nepal Drug Knowledge Base:** A mapping of the 196 NLEM Nepal 2021 molecules that are present in the DrugBank benchmark, in the form of a name-to-DrugBank-ID-to-node-number dictionary that, to our knowledge, does not currently exist.
- III. **Severity Classification:** A manually annotated severity classification of all 86 interaction types into Critical, Moderate and Low levels, each with a corresponding plain-language recommendation message that tells the user what action to take.
- IV. **Web Application (MediSafe Nepal):** A functional web interface where a doctor, pharmacist or patient can type drug names, view predicted interactions with severity levels and recommendations, and see drugs that are not listed in NLEM Nepal 2021 clearly flagged rather than hidden.
- V. **Billing Integration API:** A REST API endpoint that pharmacy billing software can call when medicines are entered during billing. The API accepts a list of drug names and returns interaction results as structured JSON, enabling automatic warning banners when dangerous combinations are detected.

In summary, the expected final product is a complete pharmacy decision-support system tailored to Nepal that can instantly warn users about dangerous drug combinations, explain why the interaction is dangerous, tell them how severe it is, and suggest what to do about it.

5. PROJECT SCHEDULE

Figure 5-1 shows the planned project timeline in the form of a Gantt chart.

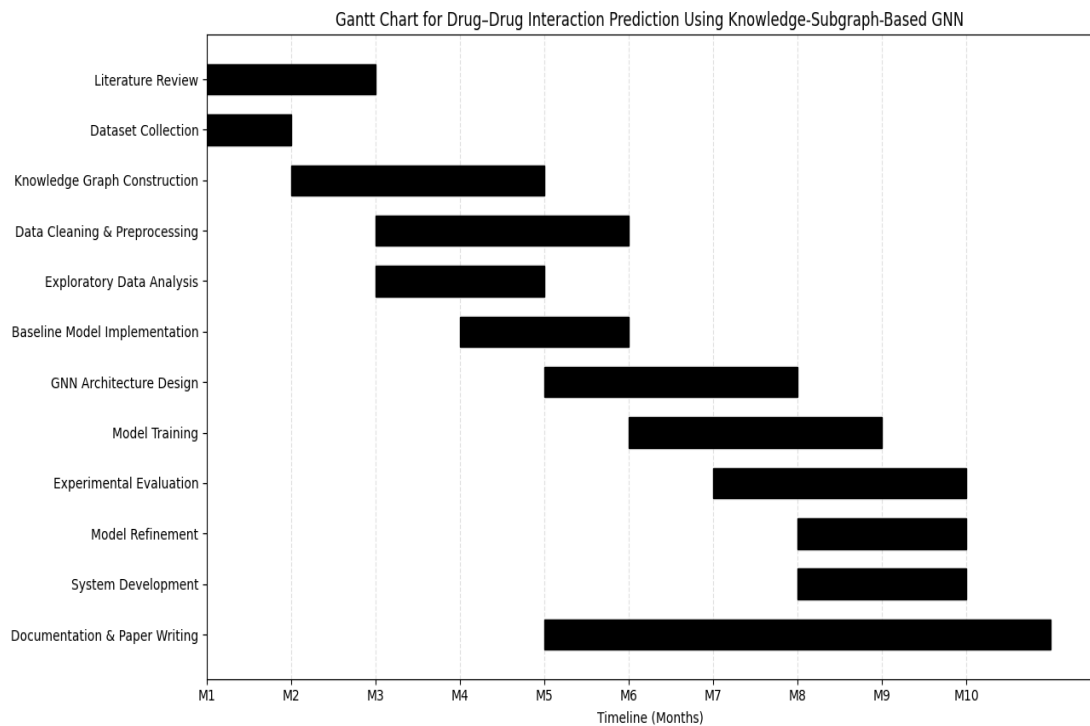


Figure 5-1.: Project Timeline (Gantt Chart)

6. PROJECT BUDGET

Table 6-1 presents the estimated budget for this project.

Table 6-1.: Project Budget

S.N.	Item	Quantity	Cost (NPR)
1	Google Colab Pro subscription (GPU for model training)	3 months	4,500
2	Web hosting and domain for deployment	1 year	5,000
3	Internet charges for data download and development	8 months	8,000
4	Printing and binding (proposal, reports, final document)	–	5,000
5	Stationery and miscellaneous	–	2,000
6	Transportation for pharmacy field visits and user testing	–	3,000
Total Estimated Cost			NPR 27,500

The total estimated cost of the project is NPR 27,500. This budget is minimal because the project relies entirely on publicly available datasets, open-source software tools and free computing resources. The benchmark DDI dataset, Hetionet knowledge graph and NLEM Nepal 2021 document are all freely accessible. Python, PyTorch, PyTorch Geometric and Streamlit are all open-source. Google Colab provides free GPU access, though the Pro subscription is budgeted for faster and more reliable training sessions.

No laboratory equipment, licensed software or paid datasets are required. Compared to discovering drug interactions through clinical trials, which would cost millions of rupees, this computational approach achieves useful predictions at a fraction of the cost.

7. FEASIBILITY ANALYSIS

This chapter examines whether the proposed system can be practically built, deployed and maintained given the available resources, time and technology.

7.1. Technical Feasibility

The core prediction model is based on a published architecture that has open-source code freely available on GitHub [3]. The code is written in Python using PyTorch and PyTorch Geometric, both of which are well documented and widely used in the research community. The benchmark dataset containing 1,710 drugs and 192,284 interactions is also publicly available and does not require any paid licence or special permission to download. The external knowledge graph Hetionet is similarly free and publicly hosted. All required software tools including Python, PyTorch, Streamlit and FastAPI are open-source and free to use.

The model can be trained on Google Colab which provides free access to a Tesla T4 GPU with 15 GB VRAM. The published training configuration uses 32-dimensional embeddings and a batch size of 256, which fits comfortably within the free Colab memory limit. Based on the published configuration, training is expected to complete within a few hours to a day on a single GPU. For inference during the web application phase, predictions for a single drug pair are expected to take on the order of a second on a standard CPU, which is fast enough for interactive screening of up to 10 drugs (45 pairs) without any specialised hardware.

The Nepal-specific mapping requires mapping NLEM Nepal 2021 drug names to DrugBank identifiers through name normalisation and matching, followed by conversion to the internal node numbers used by the dataset. This is a data-processing task that uses standard tools and does not require any advanced infrastructure. The web application can be built using Streamlit which requires minimal frontend development knowledge and can be deployed on free hosting platforms.

Based on the above, the project is technically feasible with freely available tools, data and computing resources.

7.2. Economic Feasibility

The total estimated cost of this project is minimal because almost all resources used are free. The benchmark dataset, knowledge graph and NLEM Nepal 2021 document are all publicly available at no cost. Python and all required libraries are open-source. Google Colab provides free GPU access for training. The only potential costs are Google Colab Pro subscription for faster training (approximately NPR 1,500 per month if needed), web hosting for deployment (approximately NPR 3,000 to 5,000 per year for a basic cloud server), and report printing and binding (approximately NPR 3,000 to 5,000).

The total estimated project budget is under NPR 30,000, which is affordable for a student team. Compared to the cost of developing a similar system from scratch with original laboratory data, which would require millions of rupees for clinical trials and drug testing, this approach is extremely cost-effective because it reuses existing publicly available datasets and proven algorithms.

7.3. Operational Feasibility

The target users of this system are doctors, pharmacists and informed patients. The web interface is designed to be simple: the user types drug names into a search box, clicks a button, and sees the results with colour-coded severity badges and plain-language recommendations. No knowledge of machine learning, graph theory or programming is required to use the system.

For pharmacy billing integration, the system provides a REST API that accepts drug names and returns results as JSON. Any billing software developer can integrate this API with a few lines of code. The pharmacist does not need to do anything extra because the warnings appear automatically when medicines are entered during billing.

The Nepal drug mapping can be updated when NLEM is revised in the future without retraining the model. Only the mapping files need to be edited, which is a simple task that does not require any technical expertise beyond basic JSON editing. This is possible because the underlying model already supports all 1,710 benchmark drugs; the Nepal mapping is an overlay on top of the model rather than a constraint built into it.

7.4. Schedule Feasibility

The project has a timeline of approximately 7 to 8 months with a team of four members. The work can be divided into parallel tracks: one member handles model training and validation, one member handles Nepal drug mapping and severity annotation, one member handles web application development, and one member handles testing and documentation. The most time-consuming task is expected to be setting up and training the model, which is estimated to take 2 to 3 months including debugging. The remaining tasks can proceed in parallel, making the overall timeline achievable.

Since the model architecture and code already exist as open-source, the team does not need to design or implement any new algorithm from scratch. This significantly reduces development risk and time compared to a project that requires novel algorithm development.

7.5. Legal Feasibility

All datasets used in this project are publicly available under open licences. The DrugBank benchmark data is published openly by the authors of the reference paper. Hetionet is released under a permissive open licence. NLEM Nepal 2021 is a public government document. The model code is released on GitHub as open-source.

The system is positioned as a decision-support tool, not as medical authority. It does not make any clinical decisions on its own. The final decision always remains with the doctor or pharmacist. The system does not use the word illegal for any drug. Instead it uses neutral wording such as "not listed in NLEM Nepal 2021" to avoid any legal complications. A clear disclaimer is displayed on the web interface stating that the system is a research prototype and should not replace professional medical advice.

Based on the analysis above, the project is feasible across all dimensions: technical, economic, operational, schedule and legal.

REFERENCES

- [1] B. Percha and R. B. Altman, “Informatics confronts drug-drug interactions,” *Trends in Pharmacological Sciences*, vol. 34, no. 3, pp. 178–184, 2013.
- [2] H. Jiang *et al.*, “Adverse drug reactions and correlations with drug-drug interactions: a retrospective study,” *Frontiers in Pharmacology*, vol. 13, p. 923939, 2022.
- [3] Y. Wang, Z. Yang, and Q. Yao, “Accurate and interpretable drug-drug interaction prediction enabled by knowledge subgraph learning,” *Communications Medicine*, vol. 4, no. 59, 2024.
- [4] D. S. Wishart *et al.*, “Drugbank 5.0: a major update to the drugbank database for 2018,” *Nucleic Acids Research*, vol. 46, no. D1, pp. D1074–D1082, 2018.
- [5] A. Bordes *et al.*, “Translating embeddings for modeling multi-relational data,” in *Advances in Neural Information Processing Systems*, 2013, pp. 2787–2795.
- [6] J. Yao, W. Sun, Z. Jian, Q. Wu, and X. Wang, “Effective knowledge graph embeddings based on multidirectional semantics relations for polypharmacy side effects prediction,” *Bioinformatics*, vol. 38, no. 8, pp. 2315–2322, 2022.
- [7] B. Perozzi, R. Al-Rfou, and S. Skiena, “Deepwalk: online learning of social representations,” in *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2014, pp. 701–710.
- [8] A. Grover and J. Leskovec, “node2vec: scalable feature learning for networks,” in *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 855–864.
- [9] J. Tang *et al.*, “Line: large-scale information network embedding,” in *International Conference on World Wide Web*, 2015, pp. 1067–1077.
- [10] P. Velickovic *et al.*, “Graph attention networks,” in *International Conference on Learning Representations*, 2018.
- [11] M. Zitnik, M. Agrawal, and J. Leskovec, “Modeling polypharmacy side effects with graph convolutional networks,” *Bioinformatics*, vol. 34, no. 13, pp. i457–i466, 2018.

- [12] K. Huang, C. Xiao, L. M. Glass, M. Zitnik, and J. Sun, “Skipgnn: predicting molecular interactions with skip-graph networks,” *Scientific Reports*, vol. 10, pp. 1–16, 2020.
- [13] X. Lin *et al.*, “Kggn: knowledge graph neural network for drug-drug interaction prediction,” in *International Joint Conference on Artificial Intelligence*, 2020, pp. 2739–2745.
- [14] X. Su, L. Hu, Z. You, P. Hu, and B. Zhao, “Attention-based knowledge graph representation learning for predicting drug-drug interactions,” *Briefings in Bioinformatics*, vol. 23, no. 3, p. bbac140, 2022.
- [15] K. Teru, E. Denis, and W. Hamilton, “Inductive relation prediction by subgraph reasoning,” in *International Conference on Machine Learning*, 2020, pp. 9448–9457.
- [16] Y. Yu *et al.*, “Sumggn: multi-typed drug interaction prediction via efficient knowledge graph summarization,” *Bioinformatics*, vol. 37, no. 18, pp. 2988–2995, 2021.
- [17] Y. Hong, P. Luo, S. Jin, and X. Liu, “Lagat: link-aware graph attention network for drug-drug interaction prediction,” *Bioinformatics*, vol. 38, no. 24, pp. 5406–5412, 2022.
- [18] W. Hamilton *et al.*, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems*, 2017, pp. 1024–1034.